



PALADIN
BLOCKCHAIN SECURITY

Smart Contract Security Assessment

Final Report

For LayerZero
(IzRead Differential Audit)

23 June 2026



paladinsec.co



info@paladinsec.co

Table of Contents

Table of Contents	2
Disclaimer	3
1 Overview	4
1.1 Summary	5
1.2 Scope	5
2 Diff Analysis	9
2.1 DVN.sol	9
2.2 MultiSig.sol	11
2.3 SimpleReadDVN.sol	12
2.4 DVNFeeLib.sol	14
2.5 SimpleReadExecutor.sol	20
2.6 Protocol Interface Changes	21
2.7 Interfaces & Libraries	22
2.8 ExecutorFeeLib.sol	23
2.9 Renamed Contracts: CmdLib -> ReadLib	24
3 Improvement Suggestions	27
3.1 Consider Adding Events to DVNFeeLib Admin Setters	27



Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocacy for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team.

Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

1 Overview

The Paladin LayerZero Read audit (15 November 2024) reviewed the DVN, MultiSig, CmdLib, CmdLibBase, CmdCodecV1 and OAppRead contracts at monorepo commit 80d5418. Issues were resolved at commit 56cc7e7.

This report checks the difference between 80d5418 and the current public main (9c741e7) to confirm that the audited code is logically preserved on the public LayerZero-v2 repository and to flag any security concerns in the remaining differences.

Result: The difference between 80d5418 and main is minimal. All changes fall into four categories:

1. **Rename:**

CmdLib -> ReadLib1002, CmdLibBase -> ReadLibBase, CmdCodecV1 -> ReadCmdCodecV1 and all related identifiers

2. **Audit resolution:**

address(0) signer validation in MultiSig (1zRead audit Issue #05 fix)

3. **Additive:**

Timestamp validation in DVNFeeLib, version() getters, mustExecute() in SimpleReadExecutor

4. **Cosmetic:**

Parameter name changes in protocol interfaces, comment removal

1.1 Summary

Security review of code differences between Paladin 1zRead audit commit and public LayerZero-v2 main

IzRead Audit	Paladin LayerZeroRead Report , 15 November 2024
Preliminary Commit	80d5418 (monorepo)
Resolution Commit	56cc7e7 (monorepo)
Target Commit	9c741e7 on main
Related Reports	DVN Diff Audit Report , MessageLib Diff Audit Report , EndpointV2 Diff Audit Report

1.2 Scope

All files under packages/layerzero-v2/evm/ compared between 80d5418 and main.

Production contracts – All differences

File	Changed Lines	Category	Risk
DVN.sol	6	ReadLib selector + cosmetic	None
MultiSig.sol	7	Audit Issue #05 fix	None (security fix)
DVNFeeLib.sol	63	Timestamp validation + rename	 See improvement suggestion
SimpleReadDVN.sol	23	Rename + callback pattern	None
SimpleReadExecutor.sol	10	mustExecute() addition	None
ExecutorFeeLib.sol	4	version() getter	None
IDVNFeeLib.sol	4	Additive interface	None
IExecutorFeeLib.sol	2	Additive interface	None
SupportedCmdTypes.sol	3	New constant	None
ILayerZeroComposer.sol	1	Comment removal	None
IMessageLibManager.sol	4	Param name swap (cosmetic)	None
ReadLib1002.sol (←CmdLib)	78	Rename + version bump	None
ReadLibBase.sol (←CmdLibBase)	98	Pure rename	None
ReadLib1002View.sol (←CmdLibView)	21	Rename + blank line	None
ReadCmdCodecV1.sol (←CmdCodecV1)	~60	Rename + callback refactor	None

OApp Contracts

File	Changed Lines	Category	Risk
CmdCodecV1Mock.sol	9	Rename (CmdCodecV1 -> ReadCmdCodecV1) + encode() simplification	None
LzReadCounter.sol	16	Rename + parameterized isBlockNum/blockNumOrTimestamp	None
IOAppComputer.sol	7	Refactored into IOAppComputerMap + IOAppComputerReduce	None
IOAppComputerMap.sol	7	NEW - extracted lzMap() interface	None
IOAppComputerReduce.sol	7	NEW - extracted lzReduce() interface	None
ReadCmdCodecV1.sol (oapp)	231	NEW - encoder library for oapp side	None

Files removed on main

Category	Files
DVN Adapters	ArbitrumDVNAdapterL1/L2, OptimismDVNAdapterL1/L2
Placeholder	DeadDVN.sol
Proxy	TransparentUpgradeableProxy.sol
Infrastructure	DVNDecoder, DVNDecompressor, DecompressorExtension, ExecutorDecoder, ExecutorDecompressor, LowLatWrapper, MultiCall
Mocks	DVNMock, ExecutorMock
CmdLib (renamed)	CmdLib.sol, CmdLibBase.sol, CmdLibView.sol, CmdCodecV1.sol

Overall, all security-critical logic from the Paladin 1zRead audit is the same on the public LayerZero-v2 repository at commit 9c741e7. The remaining differences consist of terminology renames, audit issue resolutions and minor additive functions. Also the single improvement suggestion (adding events) is about a pre-existing DVNFeeLib pattern, not something introduced by this diff.

2 Diff Analysis

2.1 DVN.sol

6 lines changed.

#	Change	Detail	Assessment
1	Import added	<code>import { ReadLib1002 } from "../readlib/ ReadLib1002.sol"</code>	Safe
2	<code>_shouldCheckHash()</code> extended	Added <code>ReadLib1002.verify.selector (0xab750e75)</code>	See analysis below
3-4	Named return variable	<code>return IDVNFeeLib(...).getFee(...) -> fee = IDVNFeeLib(...).getFee(...) (2 occurrences)</code>	Cosmetic

`_shouldCheckHash()` analysis:

```
function _shouldCheckHash(bytes4 _functionSig) internal pure returns (bool) {
    return
        _functionSig != IReceiveUlnE2.verify.selector &&          // 0x0223536e
        _functionSig != ReadLib1002.verify.selector &&            // 0xab750e75 (NEW)
        _functionSig != ILayerZeroUltraLightNodeV2.updateHash.selector; //
0x704316e5
}
```

`ReadLib1002.verify(bytes, bytes32, bytes32)` writes `hashLookup[keccak256(_packetHeader)][_cmdHash][msg.sender] = _payloadHash` – this is idempotent (replaying writes the same value), so skipping the hash-uniqueness check here is correct.

We verified that selector `0xab750e75` matches `keccak256("verify(bytes, bytes32, bytes32)")[:4]`.

Worth noting: In the old code at 80d5418, `CmdLib.verify()` (same selector) was NOT exempt, which means calling it through `execute()` would consume a unique hash. This change allows retrying failed verify calls, which is a security improvement.

Verdict: Clean. No bugs.



2.2 MultiSig.sol

7 lines changed.

#	Change	Detail	Assessment
1	Error declaration	<code>error MultiSig_InvalidSigner()</code>	Safe
2	Constructor guard	<code>if (signer == address(0)) revert MultiSig_InvalidSigner()</code>	Security fix
3	<code>_setSigner()</code> guard	Same <code>address(0)</code> check in the runtime setter	Security fix

We checked all signer entry points:

Entry Point	Covered?
Constructor for loop	Yes
<code>_setSigner(address, true)</code>	Yes
<code>_setSigner(address, false)</code>	N/A (removal path)

Why `address(0)` signer is high risk: In `verifySignatures()`, `lastSigner` starts at `address(0)`. The ordered-signer check `currentSigner <= lastSigner` would reject `address(0)` as equal. On top of that, while `ECDSA.tryRecover` can return `address(0)`, it always does so with a non-`NoError` error code as the error check on line 106 catches it before reaching the signer comparison.

The real issue is that a zero-address signer in `signerSet` would count toward `signerSet.length()`, which constrains the maximum allowable quorum via `_setQuorum()`. This inflates the apparent signer count, potentially letting a higher quorum be set than the number of real signers, reducing the effective security threshold. This fix resolves 1zRead audit Issue #05.

Verdict: Clean – this is an audit-driven security fix and all entry points are covered.

2.3 SimpleReadDVN.sol

23 lines changed.

#	Change	Detail	Assessment
1	Import rename	CmdCodecV1 -> ReadCmdCodecV1, CmdLib -> ReadLib1002	Rename
2	State variable rename	cmdLib -> readLib (immutable)	Rename
3	verify() target	CmdLib(cmdLib).verify(...) -> ReadLib1002(readLib).verify(...)	See below
4	Codec call	CmdCodecV1.decode(_cmd, supportedCmdTypes) -> ReadCmdCodecV1.decode(_cmd, _assertCmdTypeSupported)	See below
5	New callback	_assertCmdTypeSupported() internal view function (9 lines)	See below

verify() correctness: Both CmdLib.verify and ReadLib1002.verify have the exact same signature and implementation:

```
hashLookup[keccak256(_packetHeader)][_cmdHash][msg.sender] =  
_payloadHash. Same selector (0xab750e75), functionally equivalent.
```

Callback analysis:

```
function _assertCmdTypeSupported(  
    uint32 _targetEid,  
    bool /**_isBlockNum*/,          // IGNORED  
    uint64 /**_blockNumOrTimestamp*/, // IGNORED  
    uint8 _cmdType  
) internal view {  
    supportedCmdTypes.assertSupported(_targetEid, _cmdType);  
}
```

The callback intentionally ignores isBlockNum and blockNumOrTimestamp - SimpleReadDVN only does cmd type bitmap checking, same as before. Temporal validation is DVNFeeLib's job, not SimpleReadDVN's, so this is correct.

Verdict: Clean. Pure rename plus a design pattern change.



2.4 DVNFeeLib.sol

63 lines changed. This is the largest and most security-relevant diff, so we went through each change carefully.

2.4.1 Import Changes (2 lines)

CmdCodecV1 -> ReadCmdCodecV1 (library rename) and added SupportedCmdTypesLib import for the new CMD_V1__TIMESTAMP_VALIDATE constant. Everything is good and safe here.

2.4.2 BlockTimeConfig struct (7 lines - new)

```
struct BlockTimeConfig {  
    uint32 avgBlockTime;           // milliseconds  
    uint64 blockNum;               // reference block number  
    uint64 timestamp;              // seconds, reference timestamp  
    uint32 maxPastRetention;       // seconds  
    uint32 maxFutureRetention;     // seconds  
}
```

Storage packing: $32 + 64 + 64 + 32 + 32 = 224$ bits - fits in a single storage slot.

Type ranges are all generous enough: avgBlockTime as uint32 in milliseconds covers up to ~49.7 days per block, timestamp as uint64 in seconds covers $\sim 5.8 \times 10^{11}$ years and the retention values as uint32 cover ~136 years.

2.4.3 dstBlockTimeConfigs mapping (1 line - new)

```
mapping(uint32 dstEid => BlockTimeConfig) public dstBlockTimeConfigs;
```

This is a standard mapping added at the end of contract storage. Since DVNFeeLib inherits Ownable (not upgradeable), there are no storage collision concerns for fresh deployments.

2.4.4 setDstBlockTimeConfigs() (9 lines - new)

```
function setDstBlockTimeConfigs(  
    uint32[] calldata dstEids,  
    BlockTimeConfig[] calldata _blockConfigs  
) external onlyOwner {  
    if (dstEids.length != _blockConfigs.length) revert  
DVN_INVALID_INPUT_LENGTH();  
    for (uint256 i = 0; i < dstEids.length; i++) {  
        dstBlockTimeConfigs[dstEids[i]] = _blockConfigs[i];  
    }  
}
```

Check	Result
Access control	onlyOwner - good
Input validation - length	Array parity check is in place
Input validation - zero values	No check for avgBlockTime > 0 or timestamp > 0, but this is consistent with every other LayerZero admin setter. If misconfigured, the system fail-closes (reverts).
Event emission	No event emitted. This matches the existing DVNFeeLib pattern - see improvement suggestion below.

2.4.5 dstBlockTimeConfigs mapping (1 line - new)

```
// OLD  
CmdCodecV1.Cmd memory cmd = CmdCodecV1.decode(_cmd, supportedCmdTypes);  
  
// NEW  
ReadCmdCodecV1.Cmd memory cmd = ReadCmdCodecV1.decode(_cmd,  
_assertCmdTypeSupported);
```

The Cmd struct is identical between both codecs. The callback now passes 4 parameters (targetEid, isBlockNum, blockNumOrTimestamp, cmdType) instead of 2 (targetEid, cmdType). The function type is view, so no state mutation is possible.

2.4.6 _assertCmdTypeSupported() (37 lines - new)

```
function _assertCmdTypeSupported(  
    uint32 _targetEid, bool _isBlockNum, uint64 _blockNumOrTimestamp, uint8  
_cmdType) internal view {  
    supportedCmdTypes.assertSupported(_targetEid, _cmdType); // (A)  
    if (supportedCmdTypes.isSupported(_targetEid,  
SupportedCmdTypesLib.CMD_V1__TIMESTAMP_VALIDATE)) { // (B)  
        BlockTimeConfig memory blockCnf = dstBlockTimeConfigs[_targetEid]; // (C)  
        uint64 timestamp = _blockNumOrTimestamp; // (D)  
        if (_isBlockNum) {  
            if (_blockNumOrTimestamp > blockCnf.blockNum) { // (E)  
                timestamp = blockCnf.timestamp +  
                    ((_blockNumOrTimestamp - blockCnf.blockNum) *  
blockCnf.avgBlockTime) / 1000; // (F)  
            } else { // (G)  
                timestamp = blockCnf.timestamp - ((blockCnf.blockNum -  
_blockNumOrTimestamp) * blockCnf.avgBlockTime) / 1000; // (H)  
            }  
        }  
        if (  
            timestamp + blockCnf.maxPastRetention < block.timestamp || // (I)  
            timestamp > block.timestamp + blockCnf.maxFutureRetention // (J)  
        ) {  
            revert DVN_TimestampOutOfRange(_targetEid, timestamp);  
        }  
    }  
}
```

Arithmetic walkthrough:

(A) calls the existing bitmap check. If the cmd type is unsupported, reverts with `UnsupportedTargetEid()`.

(B) is an opt-in gate. Timestamp validation only fires when bit 2 (`CMD_V1__TIMESTAMP_VALIDATE`) is set for the target EID. Deployments with only bits 0-1 set are not affected, so this is backward compatible.

(C) loads the `BlockTimeConfig` from storage. If it was never configured, all fields default to zero (see the zero-value analysis further down).

(D) When `_isBlockNum == false`, the value is already a timestamp and gets used directly.

(E-F) Block -> Timestamp (future block):

```
timestamp = blockCnf.timestamp + ((_blockNumOrTimestamp - blockCnf.blockNum) *  
blockCnf.avgBlockTime) / 1000
```

The inner subtraction cannot underflow because it is guarded by `_blockNumOrTimestamp > blockCnf.blockNum`. For the multiplication, Solidity widens `uint32` to `uint64`, so everything stays in `uint64`. To put this in perspective, Ethereum at 12s blocks with a delta of 20M blocks gives $20M \times 12000 = 2.4 \times 10^{11}$, which is nowhere near `uint64` max. If someone did somehow push an extreme value, Solidity 0.8 checked math would just revert. The final addition is the same story- it remains as `uint64` and for any real-world timestamps and block deltas, overflow is not a concern.

(G-H) Block -> Timestamp (past block):

```
timestamp = blockCnf.timestamp - ((blockCnf.blockNum - _blockNumOrTimestamp) *  
blockCnf.avgBlockTime) / 1000
```

The inner subtraction is fine as this is in the else branch where `blockCnf.blockNum >= _blockNumOrTimestamp`. For the outer subtraction, if the computed time delta somehow exceeded `blockCnf.blockNum`, Solidity would revert. However, we confirmed this cannot actually happen with any correctly configured `BlockTimeConfig`: every real blockchain started after Unix epoch 0 (January 1, 1970), so a valid reference point always satisfies `blockCnf.timestamp > blockCnf.blockNum * avgBlockTime / 1000`. The subtraction holds even for block 0.

(I) Past retention check:

```
timestamp + blockCnf.maxPastRetention < block.timestamp
```

This checks whether the requested timestamp is more than `maxPastRetention` seconds in the past. The `uint64 + uint32` stays in `uint64` range for practical values and the comparison with `block.timestamp` (`uint256`) uses an implicit upcast – this is all fine.

(J) Future retention check:

```
timestamp > block.timestamp + blockCnf.maxFutureRetention
```

This checks whether the requested timestamp is more than `maxFutureRetention` seconds in the future. `block.timestamp + uint32` is a `uint256` computation, so no overflow risk here.

Zero-value analysis (unconfigured `BlockTimeConfig`):

If `CMD_V1__TIMESTAMP_VALIDATE` is enabled but `dstBlockTimeConfigs[targetEid]` was never set:

- All fields are 0: `avgBlockTime=0, blockNum=0, timestamp=0, maxPastRetention=0, maxFutureRetention=0`
- For `_isBlockNum=true`: `timestamp = 0 + (delta * 0) / 1000 = 0`
- Past check: `0 + 0 < block.timestamp -> TRUE -> reverts`
- End result: fail-closed. All calls revert until config is set. This matches how other LayerZero admin functions behave when uninitialized – not ideal operationally, but safe.

2.4.7 version() function (3 lines - new)

```
function version() external pure returns (uint64 major, uint8 minor) {  
    return (1, 1);  
}
```

The interface declares `view`, but the implementation uses `pure` – this is valid in Solidity since `pure` satisfies `view`. No security impact.

Verdict: No bugs. One improvement suggestion documented in Section 5.



2.5 SimpleReadExecutor.sol

10 lines changed.

New mustExecute() function:

```
function mustExecute(ExecutionParams calldata _executionParams) external payable
{
    ILayerZeroEndpointV2(endpoint).lzReceive{ value: msg.value, gas:
_executionParams.gasLimit }(
    _executionParams.origin,
    _executionParams.receiver,
    _executionParams.guid,
    _executionParams.message,
    _executionParams.extraData
    );
}
```

Check	Result
Access control	Permissionless - same as the existing execute(). By design.
Reentrancy	gasPerByte/gasPrice are not read or written by mustExecute() or execute(), so reentrancy cannot corrupt them.
msg.value	Forwarded entirely to lzReceive. If the call reverts, the whole tx reverts and ETH is returned.
Gas limit	_executionParams.gasLimit forwarded via {gas:} - same pattern as execute().
DoS/Griefing	The endpoint validates nonce/verification internally. Failed calls don't change state, so this can't be abused for DoS.
Difference from execute()	execute() wraps the call in try/catch with an lzReceiveAlert fallback. mustExecute() simply reverts on failure - intentional design for atomic execution guarantee.

Verdict: Clean. Well-designed alternative to execute().

2.6 Protocol Interface Changes

ILayerZeroComposer.sol - 1 line removed: Removed NatSpec comment @dev To ensure non-reentrancy.... No code change whatsoever.

IMessageLibManager.sol - 4 lines (parameter name swaps):

Function	Param at 80d5418	Param on main	Implementation
setDefaultReceiveLibrary	_gracePeriod	_timeout	Uses _gracePeriod
setReceiveLibraryTimeout	_expiry	_gracePeriod	Uses _expiry

Note: The interface parameter names on main do not match the implementation (which is identical at both commits). This is purely cosmetic – parameter names have no effect on ABI selectors or function signatures. The actual **MessageLibManager.sol** implementation is byte-identical between 80d5418 and main.

Verdict: Cosmetic inconsistency. No functional impact.

2.7 Interfaces & Libraries

File	Change	Assessment
IDVNFeeLib.sol	+DVN_TimestampOutOfRange(uint32,uint64) error, +DVN_INVALID_INPUT_LENGTH() error, +version() function	Additive interface alignment
IExecutorFeeLib.sol	+version() function	Additive
SupportedCmdTypes.sol	+CMD_V1__TIMESTAMP_VALIDATE = 2 constant, +comment on mapping	New constant, no effect on existing bitmaps



2.8 ExecutorFeeLib.sol

```
function version() external pure returns (uint64 major, uint8 minor) {  
    return (1, 1);  
}
```

Verdict: This is a trivial version getter. No risk.



2.9 Renamed Contracts: CmdLib -> ReadLib

2.9.1 CmdLib.sol -> ReadLib1002.sol (518 lines each)

We verified all 10 error names (LZ_CL_* -> LZ_RL_*), all struct/type names (CmdLibConfig -> ReadLibConfig) and every function name, event name, state variable, import path and comment - all consistently renamed.

Line-by-line result: pure rename with one intentional change:

Line	Old	New	Assessment
257	return (4, 0, 2); // todo: confirm this	return (10, 0, 2);	Intentional version bump. "1002" $\approx$ 10.0.2
All others	CmdLib -> ReadLib1002, LZ_CL_ -> LZ_RL_, etc.	Pure rename	Clean

2.9.2 CmdLibBase.sol -> ReadLibBase.sol (187 lines each)

Line-by-line result: 100% pure rename. All 7 error names, struct types, events and functions were consistently renamed.

Note: There is a pre-existing naming oddity: CONFIG_TYPE_CMD_LID_CONFIG / CONFIG_TYPE_READ_LID_CONFIG uses "LID" instead of "LIB". This exists in both old and new versions and is not a security issue.

2.9.3 CmdLibView.sol -> ReadLib1002View.sol (97->98 lines)

Line-by-line result: Pure rename plus 1 blank line added between import groups.

2.9.4 CmdCodecV1.sol -> ReadCmdCodecV1.sol (106->136 lines)

This is the most significant rename-refactor, so we dug into it carefully.

Function signature change:

```
// OLD
function decode(bytes calldata _cmd, SupportedCmdTypes storage
_supportedCmdTypes)
// NEW
function decode(bytes calldata _cmd, function(uint32, bool, uint64, uint8) view
_assertCmdTypeSupported)
```

Request parsing - cursor arithmetic verification:

Operation	Old	New	Bytes
Parse targetEid	cursor..cursor+4	requestCursor..requestCursor+4	4
Parse isBlockNum	(skipped)	requestCursor (1 byte)	1
Parse blockNumOrTimestamp	(skipped)	requestCursor..requestCursor+8	8
Call assertion	assertSupported(targetEid, cmdType)	_assertCmdTypeSupported(targetEid, isBlockNum, blockNumOrTimestamp, cmdType)	-
Advance cursor	cursor += requestSize	cursor += requestSize (unchanged)	requestSize

The new code introduces a separate requestCursor scoped inside a {} block. This cursor is used for parsing the individual fields but does NOT modify the main cursor, which still advances by requestSize at the end. No cursor corruption.

Compute parsing - cursor arithmetic verification:

Field	Old	New	Size
targetEid	cursor += 4	cursor += 4	4
isBlockNum	(in bulk 31)	cursor += 1	1
blockNumOrTimestamp	(in bulk 31)	cursor += 8	8
confirmations + to	(in bulk 31)	cursor += 22	22
Total	4 + 31 = 35	4 + 1 + 8 + 22 = 35	Match

Scoping blocks:

The header (cmdVersion, requestCount) is wrapped in {}. The loop now uses cmd.numEvmCallRequestV1 (set inside that block) and is functionally equivalent to the old requestCount variable. The request inner parse uses requestCursor in its own {} scope, which goes out of scope correctly.

Verdict: No bugs. Cursor arithmetic verified correct. Byte boundaries match the wire format encoder.

3 Improvement Suggestions

3.1 Consider Adding Events to DVNFeeLib Admin Setters

The new `setDstBlockTimeConfigs()` function does not emit any events on state changes. This is actually consistent with the existing DVNFeeLib codebase – `setSupportedCmdTypes()` and `setCmdFees()` do not emit events either. However, it is worth noting that the broader LayerZero codebase does follow the pattern of emitting events on admin changes: `DVN.sol` emits `SetDstConfig` and `Worker.sol` emits `SetPriceFeed`, `SetWorkerLib`, `SetDefaultMultiplierBps` and `Withdraw`.

Adding events to DVNFeeLib admin setters would improve off-chain observability and make it easier to monitor configuration changes. There is on-chain security impact either way – this is purely a best-practice suggestion.

Note: This is a pre-existing pattern in DVNFeeLib, not something newly introduced by this diff. All three admin setters (`setSupportedCmdTypes`, `setCmdFees`, `setDstBlockTimeConfigs`) lack events.





PALADIN
BLOCKCHAIN SECURITY